



Département des Technologies de l'information et de la
communication (TIC)
Filière Informatique et systèmes de communication
Orientation Sécurité informatique

Rapport détaillé

Tests effectués

Étudiant

Dylan Oliveira Ramos

Enseignant responsable

Prof. Jean-Marc Bost

Année académique

2025-2026

Yverdon-les-Bains, le 17.06.2026

Table des matières

1	Introduction	4
2	Tests effectués pour les selfies vidéo	4
2.1	Comparaison des caméras	4
2.2	Analyse des métadonnées des vidéos	5
2.3	Ajout de bruit dans les vidéos	9
2.4	Utilisation d'un modèle en 3D	9
2.5	Utilisation d'un échangeur de visage en temps réel	10
2.6	Modification du module <code>v4l2loopback</code>	10
3	Tests effectués pour la falsification de documents d'identité	13

Table des figures

Fig. 1	Caractéristiques de la caméra réelle.	4
Fig. 2	Caractéristiques de la caméra virtuelle.	4
Fig. 3	Récupération de la vidéo capturée par Facebook lors du selfie vidéo.	8
Fig. 4	Image de référence utilisée pour générer le modèle 3D.	10
Fig. 5	Modèle 3D généré par l'IA à partir de l'image de référence.	10
Fig. 6	Système recommandé pour le bon fonctionnement de Deep-Live-Cam.	10
Fig. 7	Échec de la vérification d'identité en utilisant une caméra virtuelle.	11
Fig. 8	Échec de la vérification d'identité avec une caméra réelle.	11
Fig. 9	Modification du module v4l2loopback.	12
Fig. 10	Caméra créée avec le module original.	13
Fig. 11	Caméra créée avec le module modifié.	13

1 Introduction

Certains sites candidats n'ont pas pu être contournés, soit parce qu'ils utilisent une technologie de vérification d'identité plus avancée que les autres, soit parce que la vérification d'identité est intégrée à un processus plus complexe qui rend le contournement plus difficile. Ce document présente les différents tests effectués dans l'ordre chronologique pour tenter de comprendre et de contourner les systèmes de vérification par selfie vidéo et par scan de documents d'identité.

2 Tests effectués pour les selfies vidéo

2.1 Comparaison des caméras

Il est possible que le système de vérification analyse les caractéristiques des caméras pour vérifier si celles-ci sont bien réelles. La première chose que j'ai essayé de faire est de comparer les caractéristiques de ma caméra réelle avec celles d'une caméra virtuelle en allant sur le site <https://webcamtests.com/>.

Webcam Information	
Webcam Name:	AUKEY PC—W1
Quality Rating:	23
Built-in Microphone:	None
Built-in Speaker:	None
Frame rate:	30 FPS
Stream Type:	video
Image Mode:	rgb
Webcam MegaPixels:	0.31 MP
Webcam Resolution:	640×480
Video Standard:	VGA
Aspect Ratio:	1.33
PNG File Size:	506.76 kB
JPEG File Size:	290.7 kB
Bitrate:	8.52 MB/s
Number of Colors:	30545
Average RGB Color:	
Lightness:	50.59%
Luminosity:	49.99%
Brightness:	50.59%
Hue:	330°
Saturation:	2.38%

Fig. 1. – Caractéristiques de la caméra réelle.

Webcam Information	
Webcam Name:	Virtual Cam
Quality Rating:	112
Built-in Microphone:	None
Built-in Speaker:	None
Frame rate:	31 FPS
Stream Type:	video
Image Mode:	rgb
Webcam MegaPixels:	0.31 MP
Webcam Resolution:	640×480
Video Standard:	VGA
Aspect Ratio:	1.33
PNG File Size:	637.05 kB
JPEG File Size:	330.28 kB
Bitrate:	9.98 MB/s
Number of Colors:	21969
Average RGB Color:	
Lightness:	50.78%
Luminosity:	49.51%
Brightness:	49.67%
Hue:	18°
Saturation:	16.33%

Fig. 2. – Caractéristiques de la caméra virtuelle.

Nous pouvons voir que mis à part le nom de la caméra, rien n'indique que la Fig. 2 est une caméra virtuelle, en effet, les caméras ont les mêmes megapixels (0.31 MP), résolutions (640x480) et formats d'image (1.33). D'autre part, toutes les informations en dessous de « Aspect Ratio » sont différentes, mais cela est normal car elles dépendent du contenu de la vidéo et non de la caméra elle-même. Un point intéressant à noter est que la caméra virtuelle a une fréquence d'images de 31 fps alors que la caméra réelle en a une de 30 fps, ce qui pourrait suggérer une source logicielle dont la synchronisation n'est pas liée à une horloge matérielle. Mais cela n'est pas suffisant pour conclure que la caméra virtuelle est détectée comme telle car une instabilité de mesure pourrait faire passer une valeur réelle de 30 à une valeur affichée de 31.

2.2 Analyse des métadonnées des vidéos

Étant donné que les caractéristiques des deux caméras sont plutôt similaires, je me suis penché sur l'analyse des métadonnées des vidéos diffusées. Pour cela, j'ai commencé par installer ExifTool, un outil en ligne de commande qui permet d'extraire et de modifier les métadonnées des fichiers multimédias¹.

J'ai ensuite repris une vidéo générée par le modèle HappyHorse 1.0 pour analyser ses métadonnées.

```
1 exiftool happyhorse-1-0.mp4
```

```

1 ExifTool Version Number      : 13.58
2 File Name                    : happyhorse-1-0.mp4
3 Directory                    : .
4 File Size                    : 1934 kB
5 File Modification Date/Time   : 2026:06:09 10:18:35+02:00
6 File Access Date/Time        : 2026:06:17 13:16:33+02:00
7 File Inode Change Date/Time   : 2026:06:10 14:38:42+02:00
8 File Permissions              : -rw-rw-r--
9 File Type                    : MP4
10 File Type Extension          : mp4
11 MIME Type                    : video/mp4
12 Media Data Size              : 1928320
13 Media Data Offset            : 48
14 Movie Header Version         : 0
15 Create Date                  : 0000:00:00 00:00:00
16 Modify Date                   : 0000:00:00 00:00:00
17 Time Scale                    : 1000
18 Duration                     : 5.16 s
19 Preferred Rate                : 1
20 Preferred Volume              : 100.00%
21 Preview Time                 : 0 s
22 Preview Duration             : 0 s
23 Poster Time                  : 0 s
24 Selection Time               : 0 s
25 Selection Duration           : 0 s
26 Current Time                 : 0 s
27 Next Track ID                : 3
28 Track Header Version         : 0
29 Track Create Date             : 0000:00:00 00:00:00
30 Track Modify Date            : 0000:00:00 00:00:00
31 Track ID                     : 1
32 Track Duration               : 5.04 s
33 Track Layer                   : 0
34 Track Volume                  : 0.00%
35 Image Width                  : 960
36 Image Height                 : 960
37 Graphics Mode                 : srcCopy
38 Op Color                      : 0 0 0
39 Compressor ID                : avc1
40 Source Image Width           : 960
41 Source Image Height          : 960
42 X Resolution                  : 72
43 Y Resolution                  : 72
44 Bit Depth                     : 24
45 Buffer Size                   : 0
46 Max Bitrate                  : 2936812
```

¹<https://exiftool.org/>

```

47 Average Bitrate           : 2936812
48 Video Frame Rate         : 24
49 Matrix Structure         : 1 0 0 0 1 0 0 0 1
50 Media Header Version     : 0
51 Media Create Date        : 0000:00:00 00:00:00
52 Media Modify Date        : 0000:00:00 00:00:00
53 Media Time Scale         : 24000
54 Media Duration           : 5.16 s
55 Media Language Code      : und
56 Handler Description       : SoundHandler
57 Balance                  : 0
58 Audio Format              : mp4a
59 Audio Channels            : 2
60 Audio Bits Per Sample    : 16
61 Audio Sample Rate        : 24000
62 Handler Type             : Metadata Tags
63 Major Brand               : isom
64 Minor Version            : 512
65 Compatible Brands        : isomiso2avc1mp41
AIGC                       :
{"Label": "1", "ContentProducer": "001191330106MA2CFLDG4R10001", "ProduceID": "R-
66 RI_4brMNTYm1kmZsehYuWw", "ReservedCode1": "K-
LBkc9peJ0GoxPECtYBct7vfZZgZvwy4FZrhe/DJovt3rGFEyBJ4z5WZYsl9yWMbL+j6YI/
dnc0TvjjwK1pNQvH/PdtPaJhFM6N06bmQdVF5NX2uq7G10EtaQq7CYg3wLYt1zAC4pMfsQoUzUlix
0EmtUgRhPibmjNhqlnGv8I8K2UL4nJuZWSmT2/70B9Y7whxjPn9yiVh9KsGALv9ZivDxcaEp+fjj+
67 kLWDV+iJRVqva1M9BVWyWASWEZLH6pXhuk4BXQdk/r3Y4VkCy6KPbfCgYbqjX6tBc0zUjK/
nFHw=", "ContentPropagator": "001191330106MA2CFLDG4R10001", "PropagateID": "R-
68 RI_4brMNTYm1kmZsehYuWw", "ReservedCode2": "K-
LBkc9peJ0GoxPECtYBct7vfZZgZvwy4FZrhe/DJovt3rGFEyBJ4z5WZYsl9yWMbL+j6YI/
dnc0TvjjwK1pNQvH/
PdtPaJhFM6N06bmQdVF5NX2uq7G10EtaQq7CYg3wLYt1zAC4pMfsQoUzUlix0EmtUgRhPibmjNhqln
69 Gv8I8K2UL4nJuZWSmT2/70B9Y7whxjPn9yiVh9KsGALv9ZivDxcaEp+fjj+kLWDV+iJRVqva1M9BVWy
70 WASWEZLH6pXhuk4BXQdk/r3Y4VkCy6KPbfCgYbqjX6tBc0zUjK/nFHw="}
71 Encoder                  : Lavf58.76.100
72 Image Size               : 960x960
73 Megapixels               : 0.922
74 Avg Bitrate              : 2.99 Mbps
75 Rotation                 : 0

```

Les métadonnées ci-dessus nous montrent qu'effectivement, la vidéo a été générée par une IA, en effet, le champ AIGC signifie « AI-Generated Content ». Cette mesure découle du cadre réglementaire chinois relatif à l'étiquetage obligatoire des contenus générés par l'IA, qui depuis septembre 2025, impose aux services d'IA générative d'intégrer une balise lisible par machine dans les métadonnées du fichier². Les vidéos générées par ce modèle risquent donc d'être détectées par les systèmes de vérification d'identité, mais qu'en est-il des autres modèles ?

Après avoir analysé les métadonnées de HappyHorse 1.0, j'ai analysé celles d'une vidéo générée par le modèle Kling 3.0.

```

1 exiftool kling-3-0.mp4

```

```

1 ExifTool Version Number   : 13.58
2 File Name                  : kling-3-0.mp4
3 Directory                  : .

```

²<https://www.insideprivacy.com/international/china/china-releases-new-labeling-requirements-for-ai-generated-content/>

```

 4 File Size : 2.3 MB
 5 File Modification Date/Time : 2026:06:09 10:18:07+02:00
 6 File Access Date/Time : 2026:06:13 11:51:36+02:00
 7 File Inode Change Date/Time : 2026:06:10 14:38:42+02:00
 8 File Permissions : -rw-rw-r--
 9 File Type : MP4
10 File Type Extension : mp4
11 MIME Type : video/mp4
12 Movie Header Version : 0
13 Create Date : 0000:00:00 00:00:00
14 Modify Date : 0000:00:00 00:00:00
15 Time Scale : 1000
16 Duration : 5.04 s
17 Preferred Rate : 1
18 Preferred Volume : 100.00%
19 Preview Time : 0 s
20 Preview Duration : 0 s
21 Poster Time : 0 s
22 Selection Time : 0 s
23 Selection Duration : 0 s
24 Current Time : 0 s
25 Next Track ID : 2
26 Track Header Version : 0
27 Track Create Date : 0000:00:00 00:00:00
28 Track Modify Date : 0000:00:00 00:00:00
29 Track ID : 1
30 Track Duration : 5.04 s
31 Track Layer : 0
32 Track Volume : 0.00%
33 Matrix Structure : 1 0 0 0 1 0 0 0 1
34 Image Width : 960
35 Image Height : 960
36 Media Header Version : 0
37 Media Create Date : 0000:00:00 00:00:00
38 Media Modify Date : 0000:00:00 00:00:00
39 Media Time Scale : 12288
40 Media Duration : 5.04 s
41 Media Language Code : und
42 Handler Description : VideoHandler
43 Graphics Mode : srcCopy
44 Op Color : 0 0 0
45 Compressor ID : avc1
46 Source Image Width : 960
47 Source Image Height : 960
48 X Resolution : 72
49 Y Resolution : 72
50 Compressor Name : Lavc60.31.102 h264_slapi
51 Bit Depth : 24
52 Buffer Size : 0
53 Max Bitrate : 3678734
54 Average Bitrate : 3678734
55 Video Frame Rate : 24
56 Handler Type : Metadata Tags
57 Major Brand : isom
58 Minor Version : 512
59 Compatible Brands : isomiso2avc1mp41
60 Encoder : Lavf60.16.100
61 Media Data Size : 2318369
62 Media Data Offset : 1566
63 Image Size : 960x960
64 Megapixels : 0.922
65 Avg Bitrate : 3.68 Mbps

```

66 Rotation

: 0

Nous pouvons voir que, bien que la vidéo soit générée par un modèle chinois, les métadonnées ne contiennent pas de champ AIGC. Cependant, lors d'une vérification d'identité sur Facebook par exemple, la vidéo n'est pas uploadée telle quelle, elle est capturée en temps réel par le navigateur et probablement réencodée avant d'être envoyée au serveur.

Par curiosité, nous pouvons récupérer la vidéo qui est capturée par le navigateur lors du selfie vidéo sur Facebook pour analyser ses métadonnées. Pour cela, il faut effectuer le selfie vidéo une première fois, inspecter la page du navigateur, puis aller dans l'onglet « Application ».

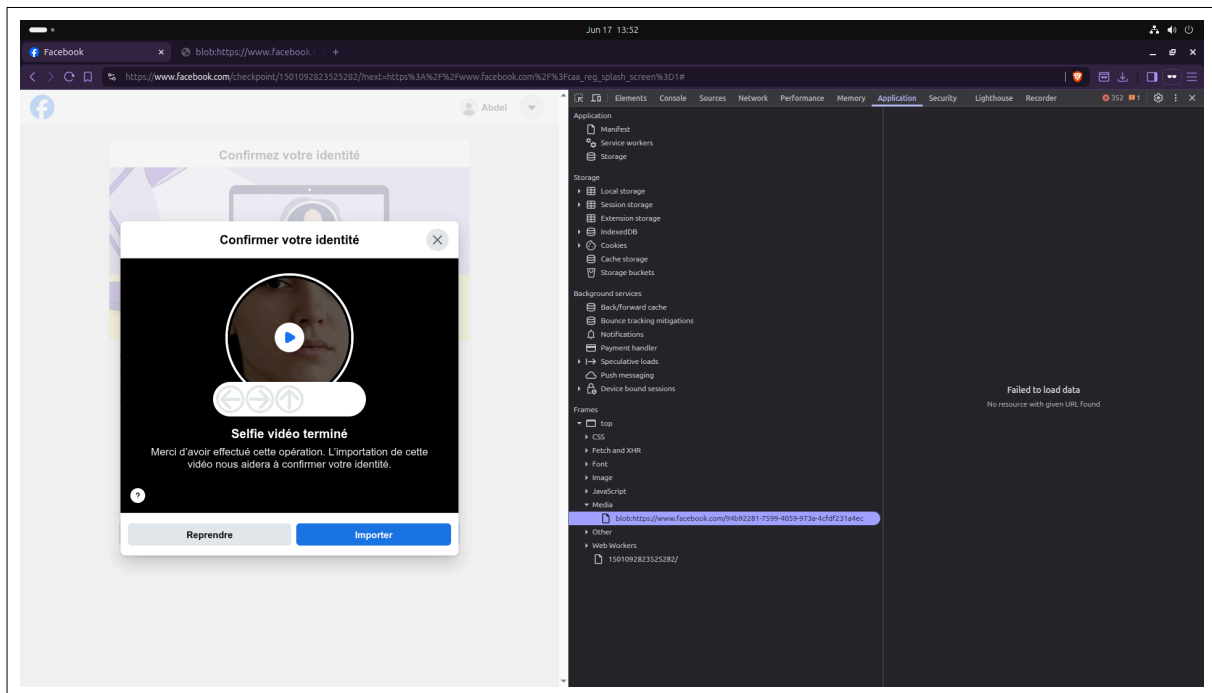


Fig. 3. – Récupération de la vidéo capturée par Facebook lors du selfie vidéo.

Nous pouvons ensuite lancer ExifTool pour analyser les métadonnées de cette vidéo.

```
1 exiftool 94b92281-7599-4059-973a-4cfd231a4ec.webm
```

```

1 ExifTool Version Number      : 13.58
2 File Name                    : 94b92281-7599-4059-973a-4cfd231a4ec.webm
3 Directory                    : .
4 File Size                    : 4.1 MB
5 File Modification Date/Time  : 2026:06:17 13:53:17+02:00
6 File Access Date/Time       : 2026:06:17 13:53:25+02:00
7 File Inode Change Date/Time  : 2026:06:17 13:53:25+02:00
8 File Permissions             : -rw-rw-r--
9 File Type                    : WEBM
10 File Type Extension         : webm
11 MIME Type                   : video/webm
12 EBML Version                : 1
13 EBML Read Version           : 1
14 Doc Type                    : webm
15 Doc Type Version            : 4
16 Doc Type Read Version       : 2
17 Timecode Scale              : 1 ms

```



```

18 Duration                : 11.46 s
19 Muxing App              : Chrome
20 Writing App             : Chrome
21 Track Number            : 1
22 Track UID               : 4d676b703b984e
23 Track Type              : Video
24 Video Codec ID          : V_VP8
25 Image Width             : 640
26 Image Height            : 480
27 Image Size              : 640x480
28 Megapixels              : 0.307

```

Nous pouvons voir que dans les métadonnées de la vidéo qui sera envoyée à Facebook, il n'y a aucune mention d'IA. En effet, la vidéo a été créée par le navigateur Chrome et encodée avec le codec VP8, les métadonnées seront donc toujours similaires quel que soit la caméra ou la vidéo utilisée.

Nous pouvons donc conclure que même si une vidéo contient des métadonnées informant que celle-ci a été générée avec de l'IA, cela n'aura aucun impact sur la vérification d'identité car elle est capturée en temps réel par le navigateur avant d'être envoyée au serveur.

2.3 Ajout de bruit dans les vidéos

Une vidéo générée par IA est souvent très « parfaite », c'est-à-dire qu'elle ne contient pas de bruit ou d'imperfections dus aux capteurs d'une caméra réelle. Il est donc possible que les systèmes de vérification repèrent les vidéos générées par IA en analysant le manque d'imperfections dans une vidéo. Pour tester cette hypothèse, j'ai ajouté du bruit³ dans les vidéos à l'aide d'un filtre FFmpeg lors des diffusions sur la caméra virtuelle.

```

1  ffmpeg_input.filter("noise", c0s=8, c0f="t+u", c1s=2, c1f="t", c2s=2, c2f="t")

```

- `c0s=8` / `c1s=2` / `c2s=2` : la luminance reçoit une valeur nettement plus élevée (8) que la chrominance (2).
- `c0f="t+u"` / `c1f="t"` / `c2f="t"` : indicateurs de bruit contrôlant la nature du bruit.
 - `t` = temporel (le motif de bruit change à chaque frame au lieu d'être figé).
 - `u` = distribution uniforme (gaussienne par défaut).

Ce filtre imite l'aspect typique du bruit d'une caméra réelle, plus fort sur la luminance, présent mais plus faible sur la chrominance et vacillant d'une frame à l'autre.

Malheureusement le résultat est le même, Facebook détecte que la vérification d'identité a échoué et désactive le compte.

2.4 Utilisation d'un modèle en 3D

Certains articles sur internet mentionnent qu'il serait possible contourner le selfie vidéo de vérification de Facebook en utilisant le modèle 3D d'une personne généré par une IA⁴. J'ai donc tenté de reproduire cette méthode en générant un modèle 3D sur le site <https://www.krea.ai/3d> et en diffusant la page web sur la caméra virtuelle de OBS Studio.

³<https://ffmpeg.org/ffmpeg-filters.html#noise>

⁴https://procpa.media/articles/kak-proiti-video-selfi-facebook/#%D0%A1%D0%BF%D0%BE%D1%81%D0%BE%D0%B1_%D1%82%D1%80%D0%B5%D1%82%D0%B8%D0%B9_%E2%80%93%D1%87%D0%B5%D1%80%D0%B5%D0%B7_3D_%D0%BC%D0%BE%D0%B4%D0%B5%D0%BB%D1%8C

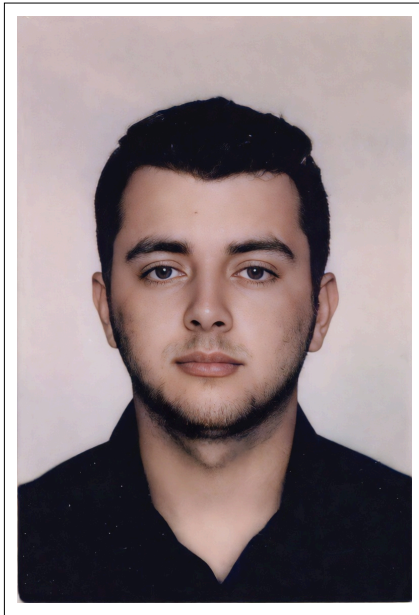


Fig. 4. – Image de référence utilisée pour générer le modèle 3D.

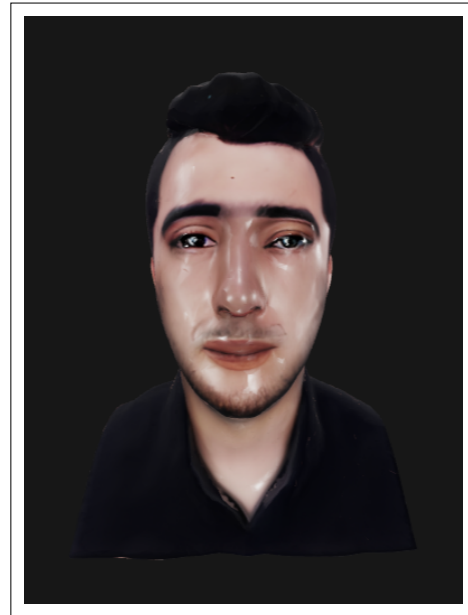


Fig. 5. – Modèle 3D généré par l'IA à partir de l'image de référence.

Résultat : [videos/08-attaques-non-reussies/facebook-3d.mp4](https://www.facebook.com/videos/08-attaques-non-reussies/facebook-3d.mp4)

Sans surprise, cette méthode ne fonctionne pas non plus, Facebook détecte que la vérification d'identité a échoué et désactive le compte.

2.5 Utilisation d'un échangeur de visage en temps réel

Plutôt que de générer ou d'éditer une vidéo, j'ai tenté d'utiliser Deep-Live-Cam, un échangeur de visage en temps réel permettant de remplacer son visage par celui d'une autre personne⁵. L'outil est un programme Python qui tourne sur la machine locale et qui s'appuie sur des modèles pré-entraînés comme *inswapper* pour l'échange de visages et *GFPGAN* pour l'amélioration des visages⁶.

Le problème est que pour utiliser cet outil, il faut avoir un système très puissant avec une bonne carte graphique, comme le montre la Fig. 6 ci-dessous.

Recommended System	Operating System: Windows 10 onwards GPU: Nvidia 2060, 3060 or higher (Nvidia GPU with 6GB VRAM) or any AMD GPU with 6GB VRAM Software Installed: CUDA 13.0 or higher if you are using Nvidia GPU
---------------------------	---

Fig. 6. – Système recommandé pour le bon fonctionnement de Deep-Live-Cam.

J'ai donc dû faire appel à un ami qui possède une machine avec une carte graphique NVIDIA RTX 3080 pour tester cet outil.

2.6 Modification du module `v4l2loopback`

Lorsque nous tentons de contourner la vérification d'identité du site Parship, nous voyons le message d'erreur ci-dessous.

⁵<https://github.com/hacksider/deep-live-cam>

⁶<https://huggingface.co/hacksider/deep-live-cam/tree/main>



Fig. 7. – Échec de la vérification d'identité en utilisant une caméra virtuelle.

Contrairement à Facebook, Parship nous dit pourquoi la vérification a échoué, nous pouvons donc en déduire que le système analyse également le type de caméra qui est utilisé. J'ai donc essayé de refaire la vérification, mais cette fois-ci avec une caméra réelle pour voir si le résultat était différent. Pour cela, j'ai affiché l'image de la personne fictive sur mon écran et j'ai filmé cette image avec ma caméra.

Résultat : [videos/08-attaques-non-reussies/parship-real-camera.mp4](#)

Le résultat est bien différent, cela confirme donc que le système de vérification de Parship analyse le type de caméra utilisé et que la diffusion d'une vidéo sur une caméra virtuelle est détectée et bloquée.

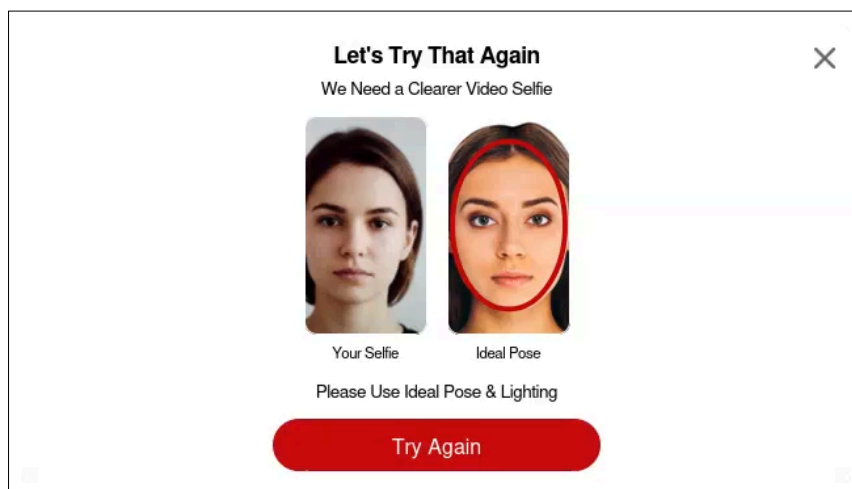


Fig. 8. – Échec de la vérification d'identité avec une caméra réelle.

Suite à cette découverte, j'ai cherché un moyen de faire croire au système de vérification que la caméra virtuelle était une caméra réelle. Pour cela, j'ai essayé de modifier le code source du module `v4l2loopback` utilisé pour créer la caméra virtuelle pour qu'il n'affiche pas les caractéristiques d'une caméra virtuelle mais celles d'une caméra réelle.

Étape 1 : cloner le dépôt GitHub du projet `v4l2loopback`.

```
1 git clone git@github.com:v4l2loopback/v4l2loopback.git
2 cd v4l2loopback
```

Étape 2 : ouvrir le fichier `v4l2loopback.c` et modifier les chaînes de caractères affichées par des `snprintf`.

```

dylan@udesktop:~/Documents/v4l2loopback$ git diff
diff --git a/v4l2loopback.c b/v4l2loopback.c
index 43ecbff..daf5934 100644
--- a/v4l2loopback.c
+++ b/v4l2loopback.c
@@ -74,7 +74,7 @@
     KERNEL_VERSION(V4L2LOOPBACK_VERSION_MAJOR, V4L2LOOPBACK_VERSION_MINOR, \
                     V4L2LOOPBACK_VERSION_BUGFIX)

-MODULE_DESCRIPTION("v4l2 loopback video device");
+MODULE_DESCRIPTION("AUKEY PC-W1: AUKEY PC-W1 video device");
MODULE_AUTHOR("Vasily Levin, "
              "Iohannes m zmoelnig <zmoelnig@iem.at>,"
              "Stefan Diewald,")
@@ -905,10 +905,10 @@ static int vidioc_querycap(struct file *file, void *fh,
    int device_nr = v4l2loopback_get_vdev_nr(dev->vdev);
    __u32 capabilities = V4L2_CAP_STREAMING | V4L2_CAP_READWRITE;

-    strscpy(cap->driver, "v4l2 loopback", sizeof(cap->driver));
-    snprintf(cap->card, sizeof(cap->card), "%s", dev->card_label);
+    strscpy(cap->driver, "uvcvideo", sizeof(cap->driver));
+    snprintf(cap->card, sizeof(cap->card), "AUKEY PC-W1: AUKEY PC-W1");
    snprintf(cap->bus_info, sizeof(cap->bus_info),
-           "platform:v4l2loopback-%03d", device_nr);
+           "usb-0000:00:14.0-13");

    if (dev->announce_all_caps) {
        capabilities |= V4L2_CAP_VIDEO_CAPTURE | V4L2_CAP_VIDEO_OUTPUT;
@@ -2862,7 +2862,7 @@ static int v4l2_loopback_add(struct v4l2_loopback_config *conf, int *ret_nr)
    }
    snprintf(dev->v4l2_dev.name, sizeof(dev->v4l2_dev.name),
-           "v4l2loopback-%03d", nr);
+           "AUKEY PC-W1: AUKEY PC-W1");

    err = v4l2_device_register(NULL, &dev->v4l2_dev);
    if (err)
dylan@udesktop:~/Documents/v4l2loopback$ |

```

Fig. 9. – Modification du module v4l2loopback.

Étape 3 : recompiler et réinstaller le module.

```

1 make
2 sudo make install
3 sudo depmod -a

```

Étape 4 : créer une nouvelle caméra virtuelle.

```

1 sudo modprobe v4l2loopback devices=1 video_nr=0 card_label="AUKEY PC-W1: AUKEY
   PC-W1" exclusive_caps=1

```

```

dylan@udesktop:~$ v4l2-ctl --device=/dev/video1 --info
Driver Info:
    Driver name      : v4l2 loopback
    Card type        : AIFRB Virtual Camera
    Bus info         : platform:v4l2loopback-001
    Driver version    : 7.0.0
    Capabilities     : 0x85200002
        Video Output
        Read/Write
        Streaming
        Extended Pix Format
        Device Capabilities
    Device Caps      : 0x05200002
        Video Output
        Read/Write
        Streaming
        Extended Pix Format
dylan@udesktop:~$ |

```

Fig. 10. – Caméra créée avec le module original.

```

dylan@udesktop:~$ v4l2-ctl --device=/dev/video2 --info
Driver Info:
    Driver name      : uvcvideo
    Card type        : AUKEY PC-W1: AUKEY PC-W1
    Bus info         : usb-0000:00:14.0-13
    Driver version    : 7.0.0
    Capabilities     : 0x85200002
        Video Output
        Read/Write
        Streaming
        Extended Pix Format
        Device Capabilities
    Device Caps      : 0x05200002
        Video Output
        Read/Write
        Streaming
        Extended Pix Format
dylan@udesktop:~$ |

```

Fig. 11. – Caméra créée avec le module modifié.

Cependant, malgré toutes ces modifications, j'obtiens le même résultat, Parship détecte toujours que la caméra utilisée est une caméra virtuelle.

3 Tests effectués pour la falsification de documents d'identité